The Shortest Path Problem

Bellman-Ford, Dijkstra, generalizations

David Pisinger

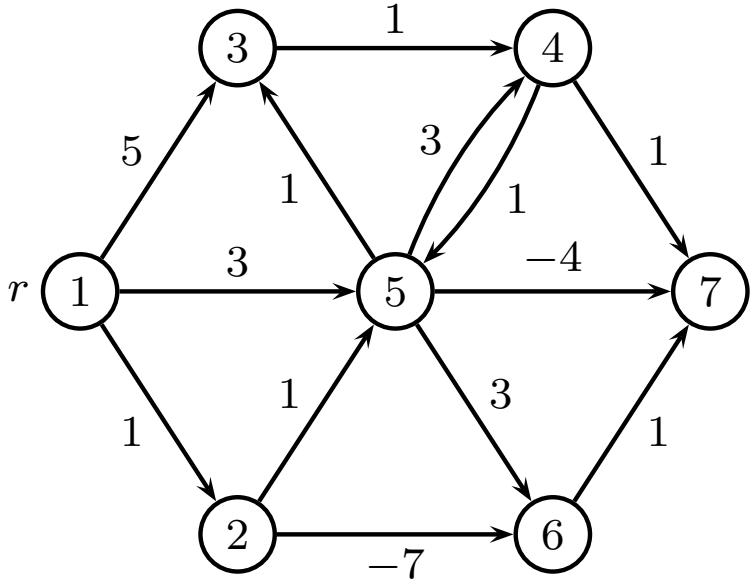
pisinger@man.dtu.dk

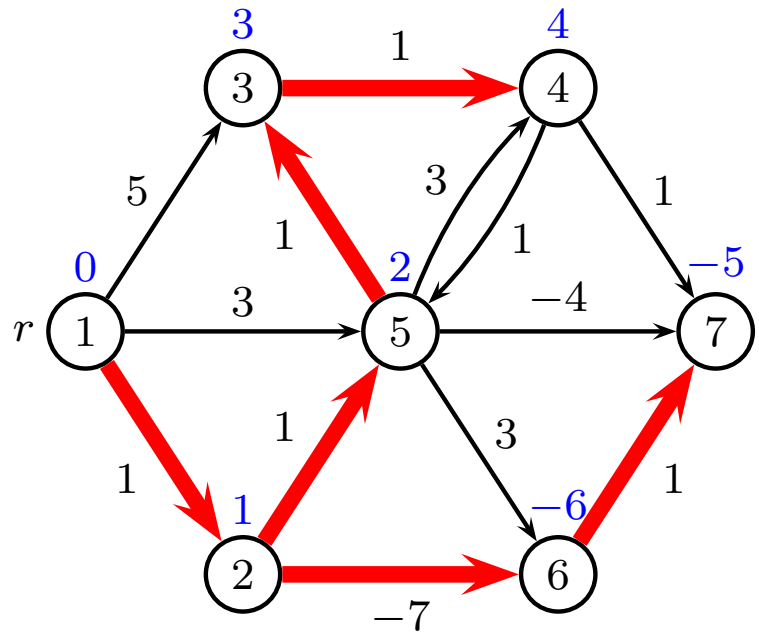
DTU Management

Technical University of Denmark

The Shortest Path Problem

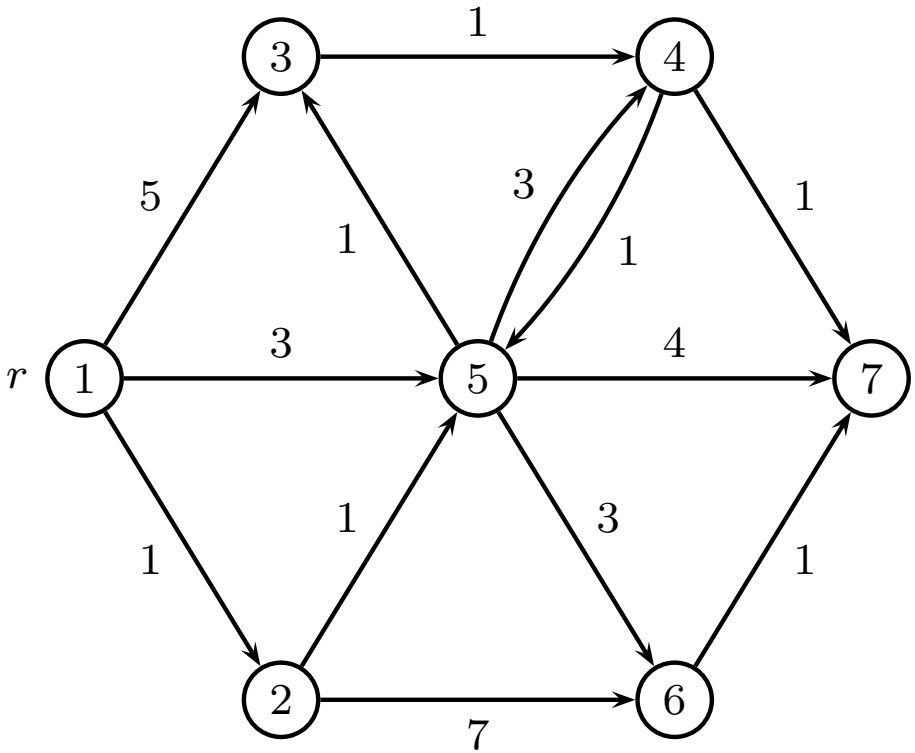
- Given a **directed** network $\mathcal{G} = (V, E, w)$ where $w \in \mathbb{R}$ (n nodes, m edges)
- Furthermore, a **source vertex** r is given.
- **Objective:** Find for each $v \in V$ a shortest (with respect to w) directed path from r to v (if such one exists).
- **No negative cycles** reachable from r

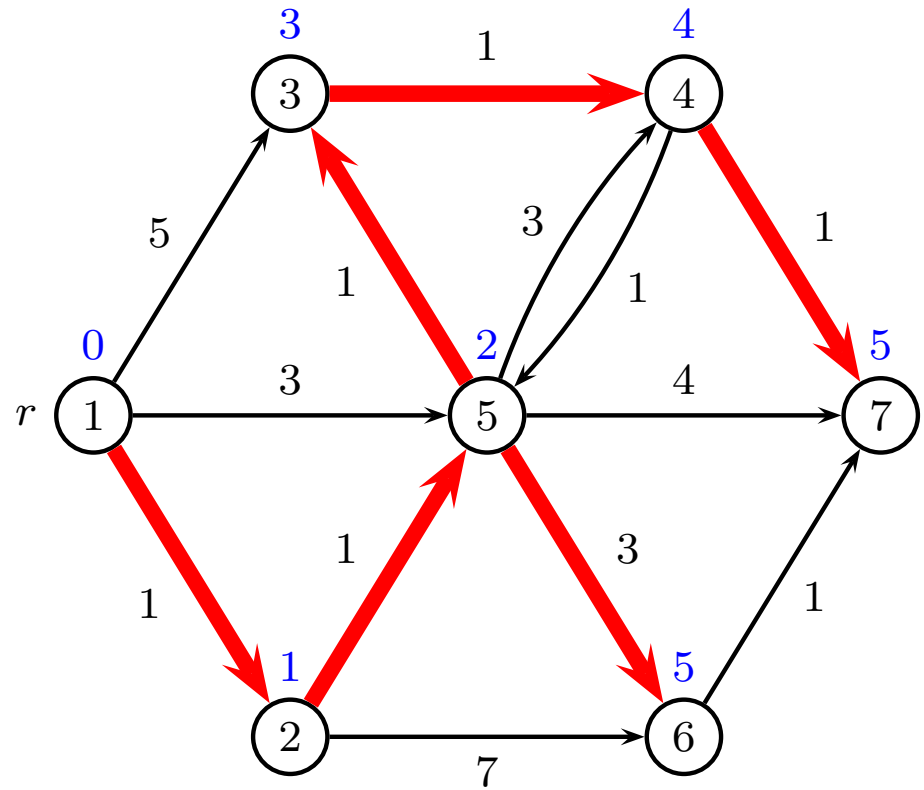




Example

(another example with positive edges)

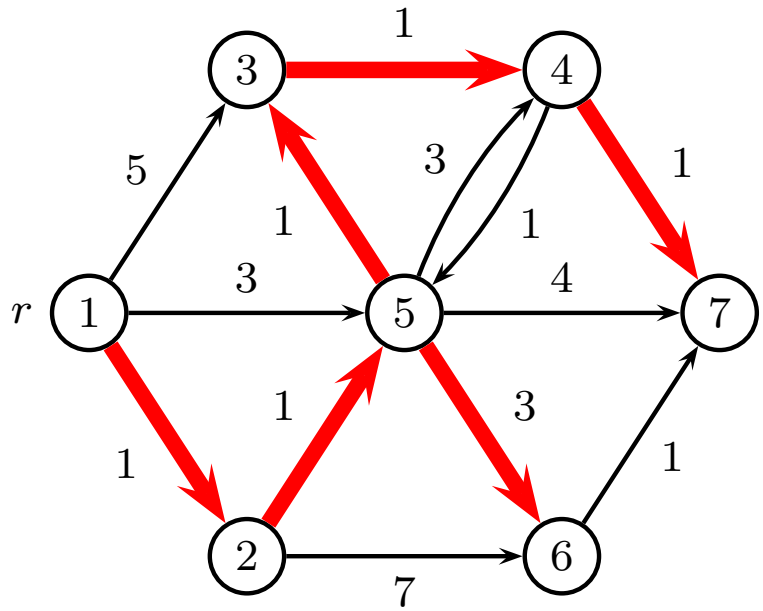




Shortest path tree rooted at r

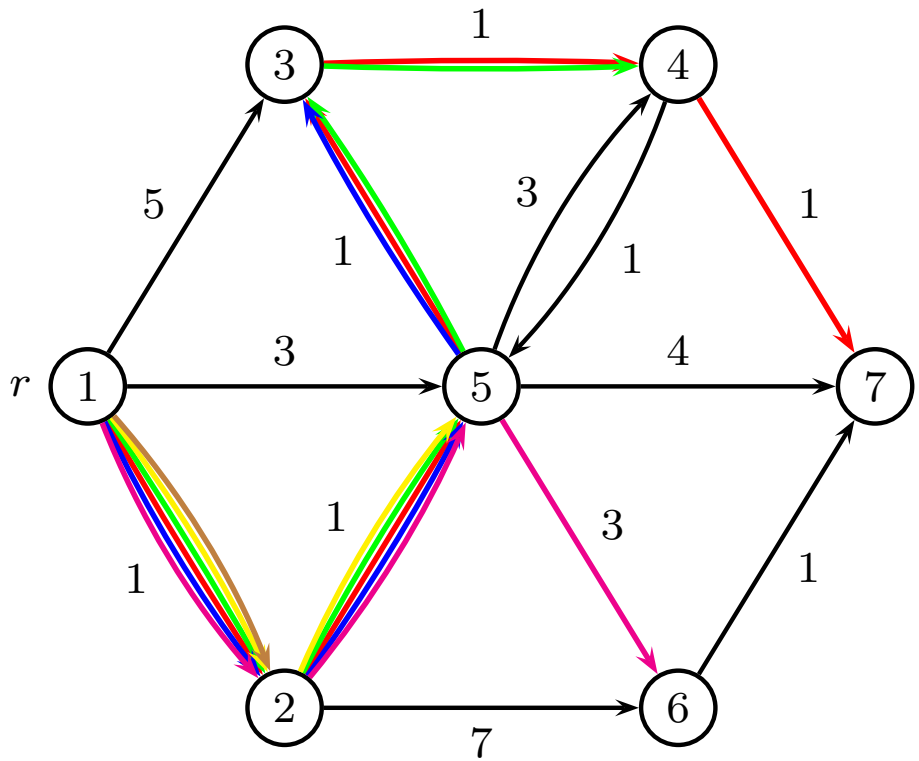
Integer Programming Formulation

- Tree rooted at r. Look at the **number** of paths leaving a vertex vs. the **number** of paths entering a vertex.
  - Vertex r must have $n - 1$ paths leaving the vertex.
  - For any other vertex, the number of paths entering the vertex must be exactly 1 larger than the number of paths leaving the vertex.
- Let x_e denote the **number** of paths using edge $e \in E$.



Example

x_e denotes number of paths using edge e



Mathematical Model

Shortest path from r to all nodes v
IP model:

$$\begin{aligned}
\min \quad & \sum_{(i,j)\in E} w_{ij}x_{ij} \\
\text{s.t.} \quad & \sum_{i\in V} x_{ri} - \sum_{i\in V} x_{ir} = n-1 \\
& \sum_{i\in V} x_{ji} - \sum_{i\in V} x_{ij} = -1 \qquad j \neq r \\
& x_{ij} \in \mathbb{Z}_+ \qquad (i,j) \in E
\end{aligned}$$

LP-problem gives IP-solution, since matrix is TU
Can be solved with CPLEX.
Will show more efficient algorithm

Mathematical model for $r \to t$

Shortest path from r to t:

$$
\begin{aligned}
\min \quad & \sum_{(i,j)\in E} w_{ij}x_{ij} \\
\text{s.t.} \quad & \sum_{i\in V} x_{ri} - \sum_{i\in V} x_{ir} = 1 \\
& \sum_{i\in V} x_{ti} - \sum_{i\in V} x_{it} = -1 \\
& \sum_{i\in V} x_{ji} - \sum_{i\in V} x_{ij} = 0 \qquad j \neq \{r,t\} \\
& x_{ij} \in \{0,1\} \qquad\qquad (i,j) \in E
\end{aligned}
$$

LP-problem gives IP-solution, since matrix is TU

Dual problem for $r \to t$

Objective $\max y_r - y_t$ equivalent to $\min y_t - y_r$

$$\begin{array}{lll} \min & y_t - y_r & \\ \text{s.t.} & y_j - y_i \leq w_{ij} & (i,j) \in E \\ & y_i \in \mathbb{R} & i \in V \end{array}$$

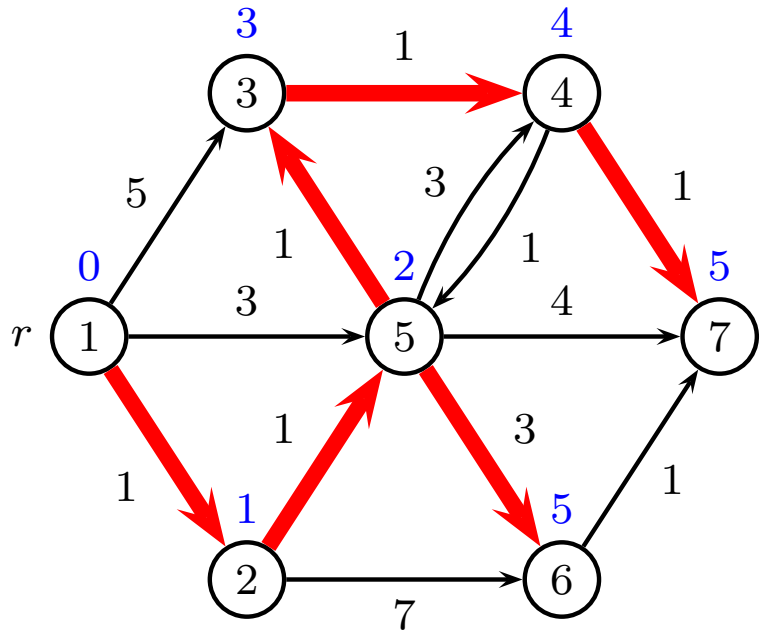
Many equally good solutions, so fix $y_r = 0$

Feasible potentials

- Consider an n-vector $y = y_1, \ldots, y_n$ (**potentials**)
- If y satisfies that $y_r = 0$ and

$$\forall (i,j) \in E : y_i + w_{ij} \geq y_j \qquad \text{(no shortcuts)}$$

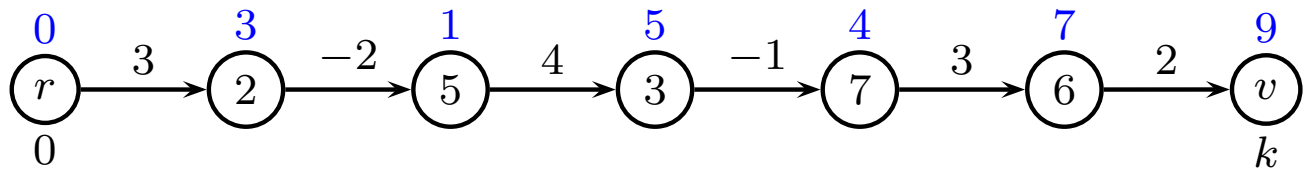
  then y is called a **feasible** potential.
- Informally: potentials y_v are possible distances from r to v. Potentials are feasible when no more shortcuts are possible



Feasible potentials II

- If P is a path from r to $v \in V$, and, if y is a feasible potential, then $w_P \geq y_v$:

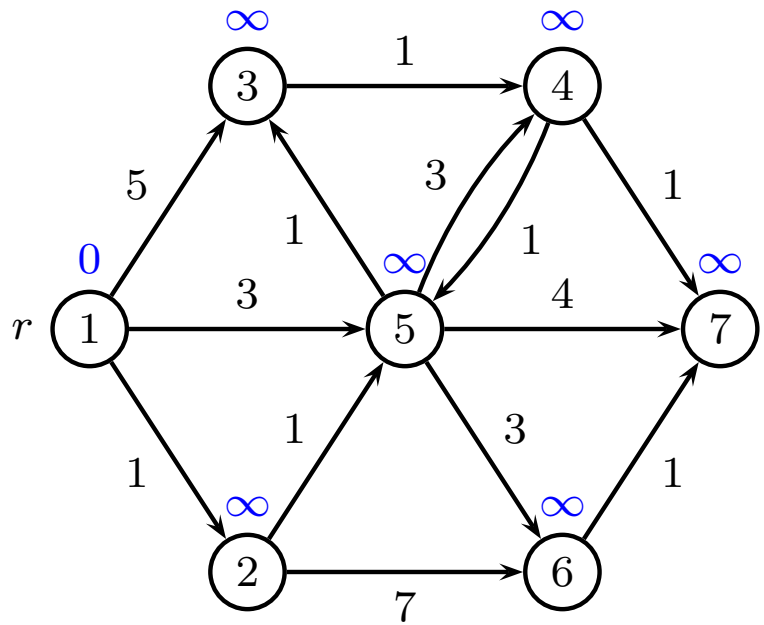
$$\begin{aligned} w_P &= \sum_{i=1}^{k} w_{e_i} \\ &\geq \sum_{i=1}^{k} (y_{v_i} - y_{v_{i-1}}) \\ &= y_{v_k} - y_{v_0} \\ &= y_v \end{aligned}$$

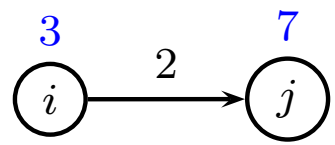


- y_v is smaller or equal to length of any path r to v

Basic algorithmic idea

- Start with the potential $y_r = 0$ and $y_i = \infty,\ i \in V \setminus \{r\}$
- Check for each edge if the potential is feasible
- If YES - Stop - the potentials identify shortest paths
- If an edge (i, j) violates feasibility condition, update y_j. This is sometimes called "correct (i, j)" or "relax (i, j)"

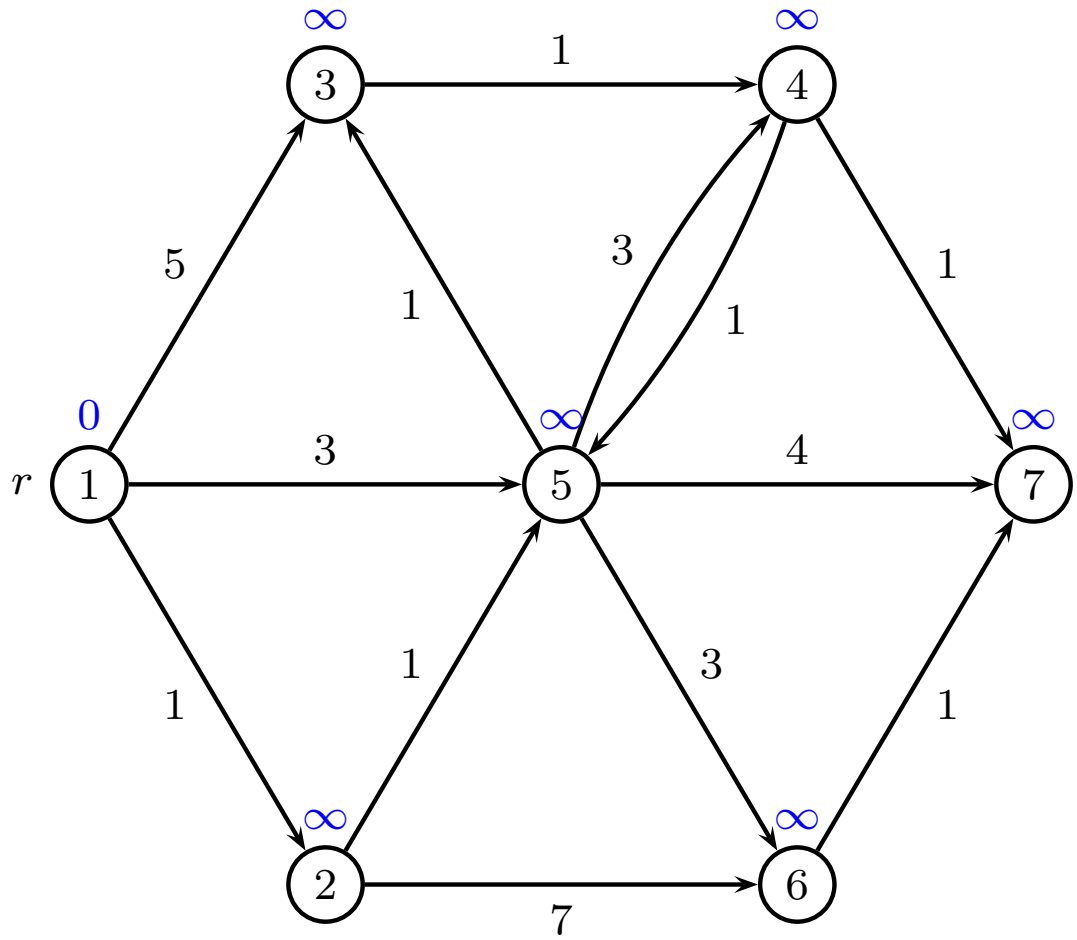




Bellman-Ford's Shortest Path Algorithm

```
1 y_r ← 0; y_i ← ∞ for all other i;
2 p_r ← 0; p_i ← NIL for all other i;
3 for k = 1 to n − 1 do
4     for (i, j) ∈ E do
5         if y_i + w_ij < y_j then                    /* correct (i, j) */
6             y_j ← y_i + w_ij;
7             p_j ← i;
8 for (i, j) ∈ E do   /* check if negative cycle */
9     if y_i + w_ij < y_j then
10        negative cycle found, follow p_i to find the cycle
```

Example



Complexity of Bellman-Ford's Algorithm

A worst-case time complexity analysis leads to the following conclusions:

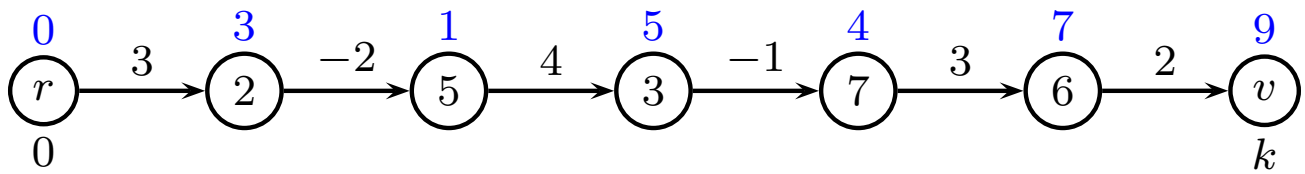
- Initialization: $O(n)$.
- Outer loop: $(n-1)$ times.
- In the loop: each edge is considered one time: $O(m)$.

In total: $O(nm)$.

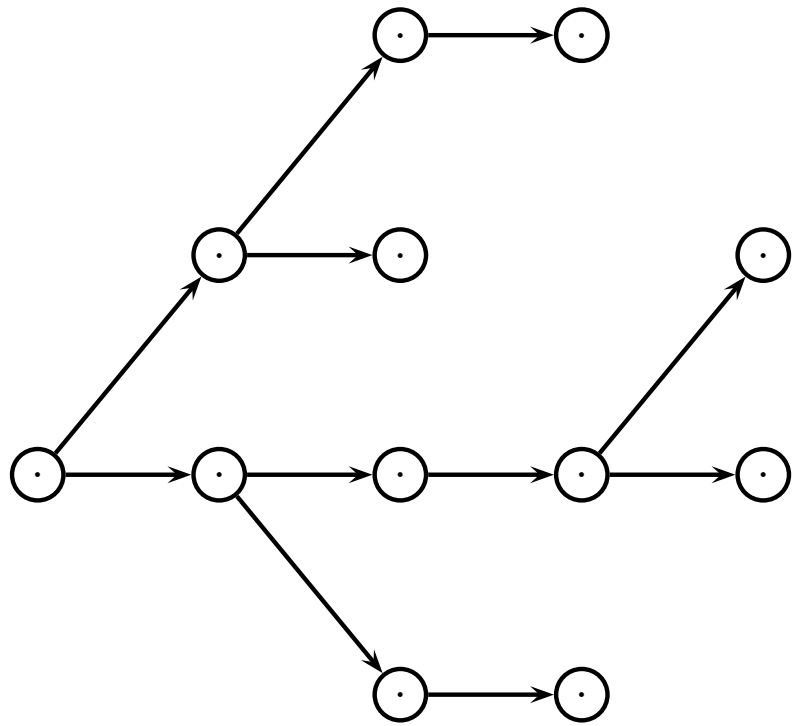
Correctness of Bellman-Ford's Algorithm

Proof by induction

- The induction hypothesis is: After iteration k of the main loop, y_i contains the length of any shortest path with *at most* k edges from 1 to i for any $i \in V$.
- For the **base case** $k = 0$ the induction hypothesis is trivially fulfilled. ($y_r = 0 =$ shortest path from r to r).
- For the **inductive step**, we assume that $y_{v_{k-1}}$ = shortest path from r to v_{k-1} after the $(k-1)$'st pass. Then (v_{k-1}, v_k) is corrected in the k'th iteration. So y_{v_k} = shortest path from r to v_k.



Correctness of Bellman-Ford's Algorithm



$0 \quad \ldots \quad k-1 \quad k$

Correctness of Bellman-Ford's Algorithm

- If **no negative length cycles**, a shortest path containing at most $(n-1)$ edges exists for each $v \in V$
- A **negative length cycle** can be discovered by an extra iteration in the main loop. If at least one y_i changes, there is a negative length circuit.

Introduction to Dijkstra's Algorithm

If we assume all edge weights $w_e \geq 0$ are non-negative we can derive a more efficient algorithm.

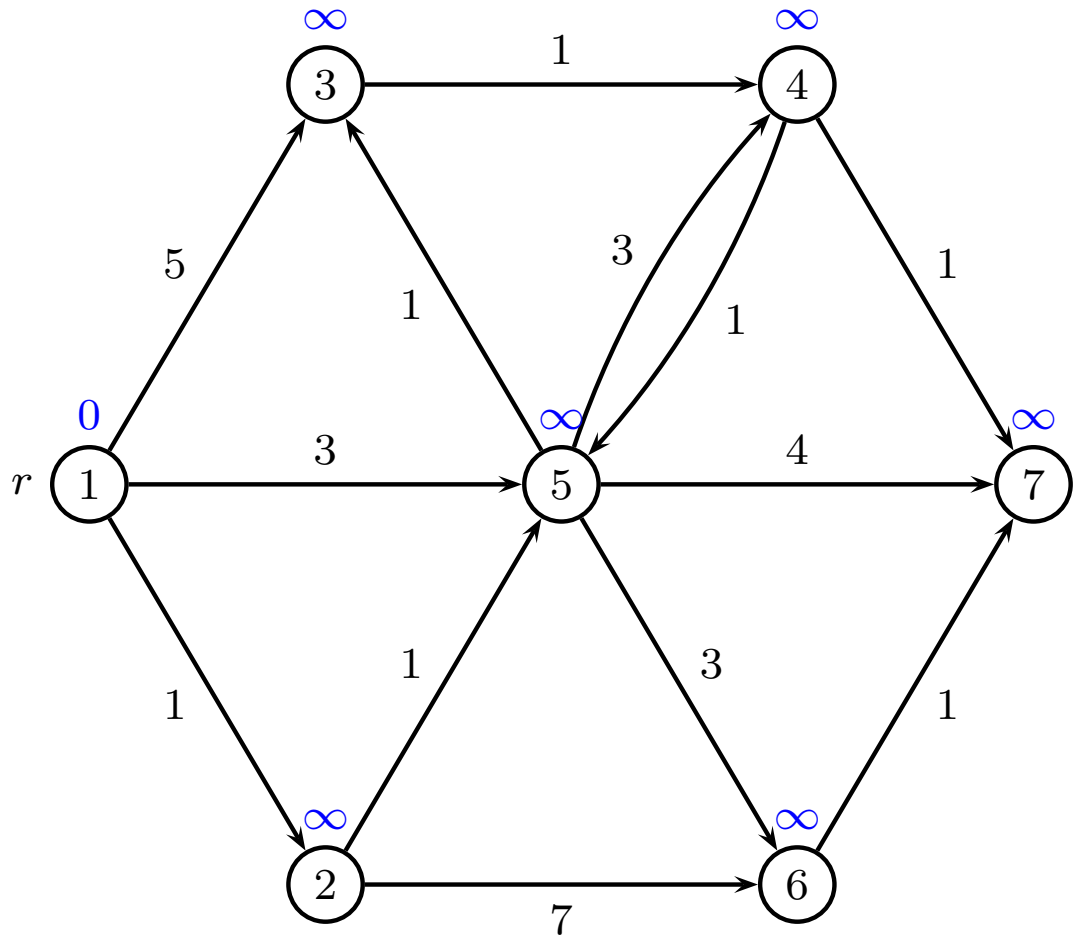
- Only consider each node once
- Add node v to set T when considered
- Always select the node $v \notin T$ with smallest potential
- Correct all outgoing edges from v

Greedy method

Dijkstra's Algorithm for Shortest Path

```
1 T ← ∅;
2 y_r ← 0; y_i ← ∞ for all other i;
3 while V \ T ≠ ∅ do
4     select an i ∈ V \ T minimizing y_i;
5     for {(i, j) ∈ E : j ∉ T} do
6         if y_i + w_ij < y_j then
7             y_j ← y_i + w_ij;
8             p_j ← i;
9     T ← T ∪ {i};
```

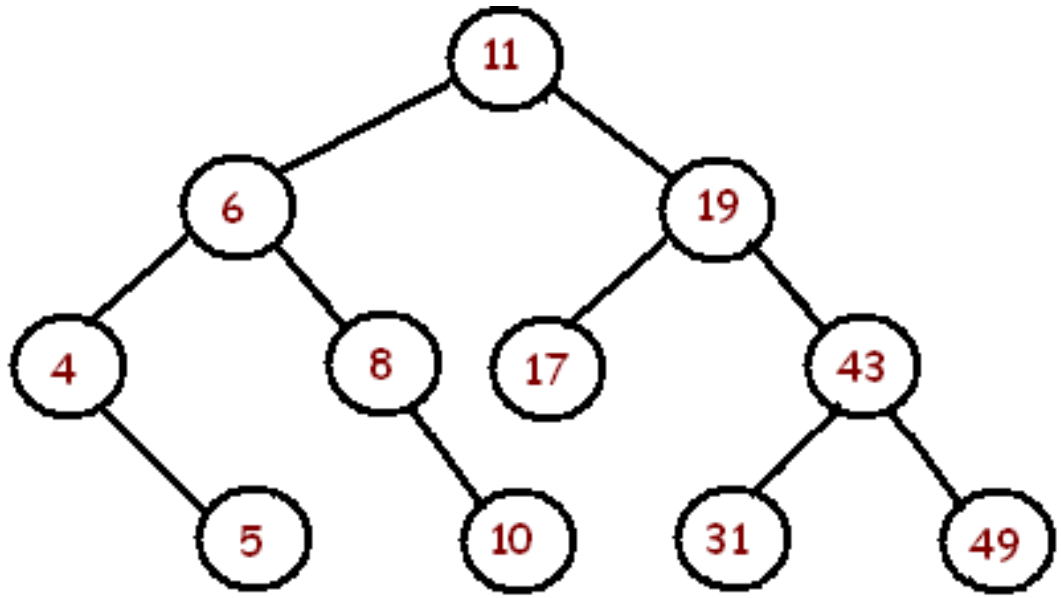
Example



Complexity of Dijkstras Algorithm

- n numbers $\{y_i\}$ in data structure
- n times find minimum in time $O(\log n)$
- m times correct edge (i, j) in time $O(1)$
- m times update y_j by delete/insert in time $O(\log n)$

In total: $O(m \log n)$



Correctness of Dijkstras Algorithm

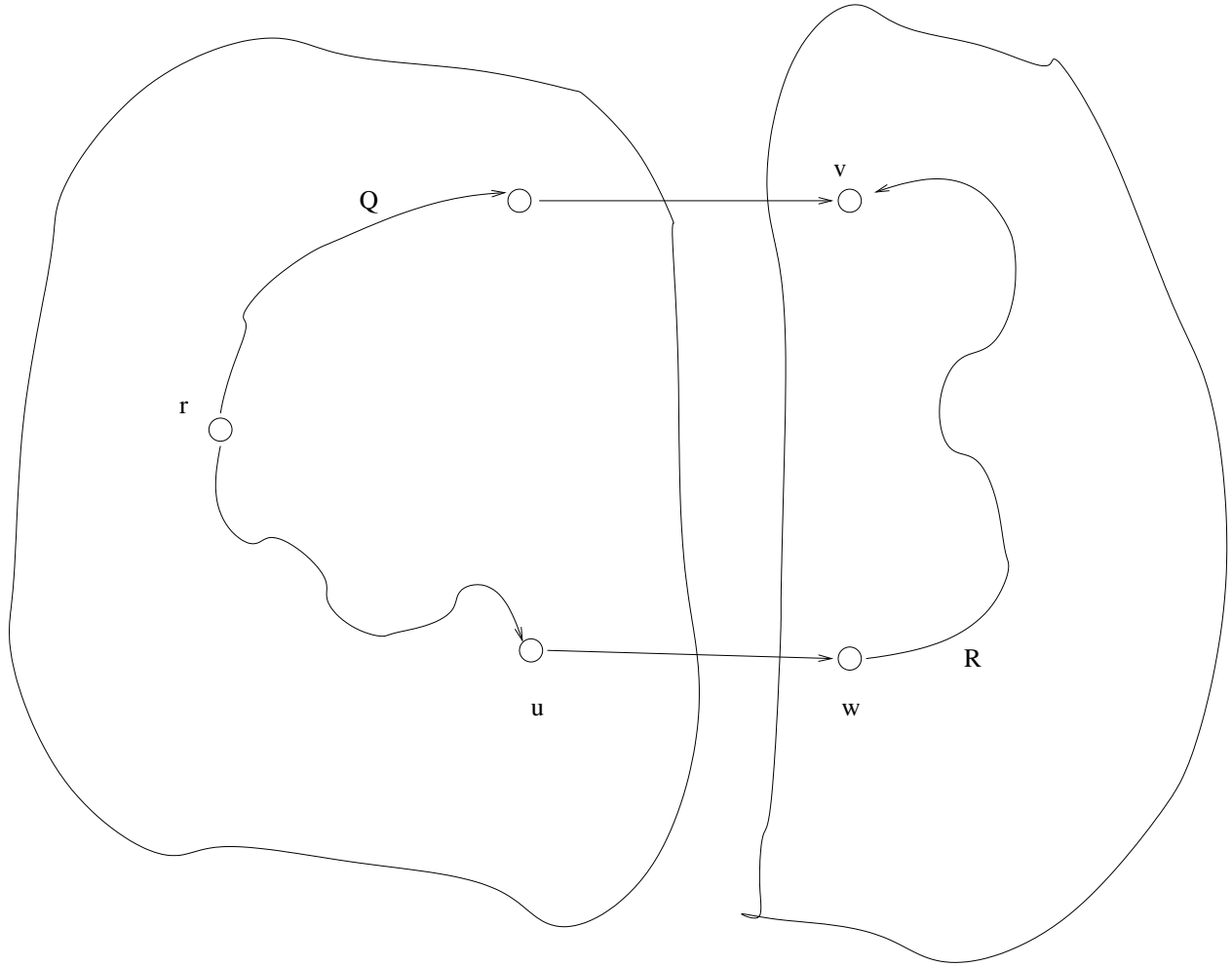
When v is added to T then $y_v = d_v$

- y_j is upper bound on distance to j
- d_j is correct shortest distance to j

Proof by contradiction

- Let v be first node where $y_v \neq d_v$ when v is added to T
- For the root $y_r = d_r = 0$ so $v \neq r$
- Another path R from r to v must be shorter than y_v

Correctness of Dijkstras Algorithm III



Correctness of Dijkstras Algorithm

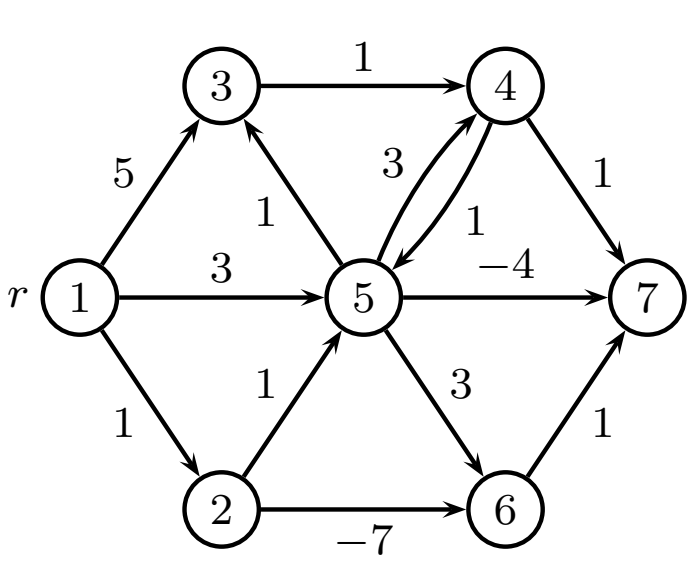
- $y_v > d_v$ since algorithm failed
- $y_v \leq y_w$ sine v was selected before w
- $y_u = d_u$ since $u \in T$
- $y_w = d_w$ since when u was considered, edge (u, w) was updated
- $d_v \geq d_w$ since edge weights are positive

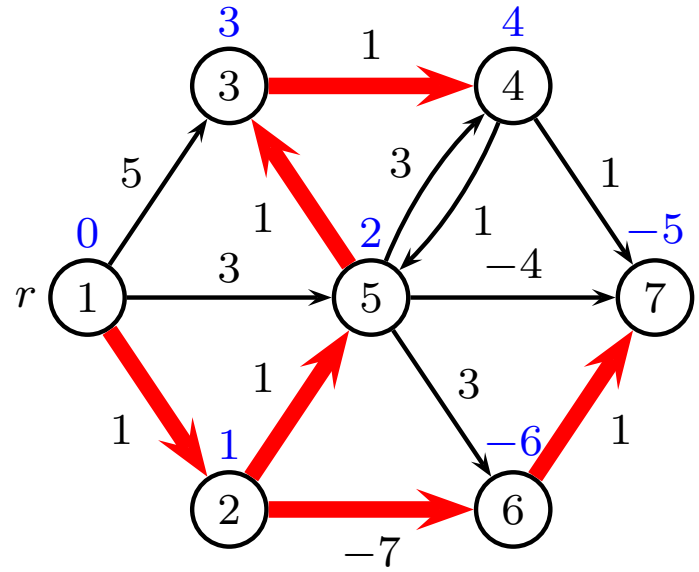
Contradiction

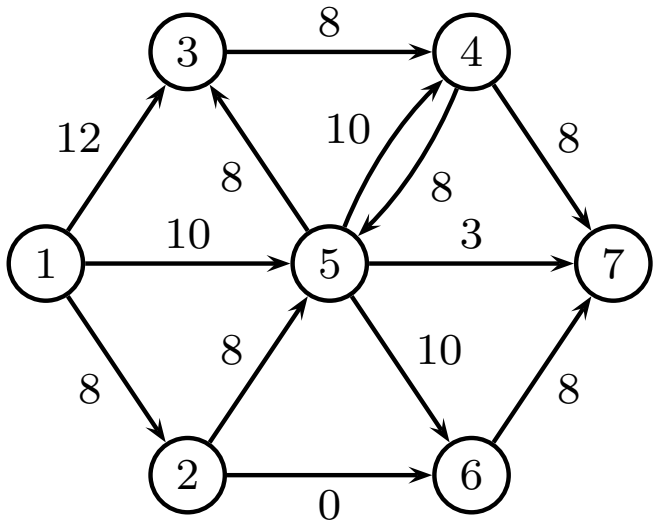
$$d_v < y_v \leq y_w = d_w \leq d_v$$

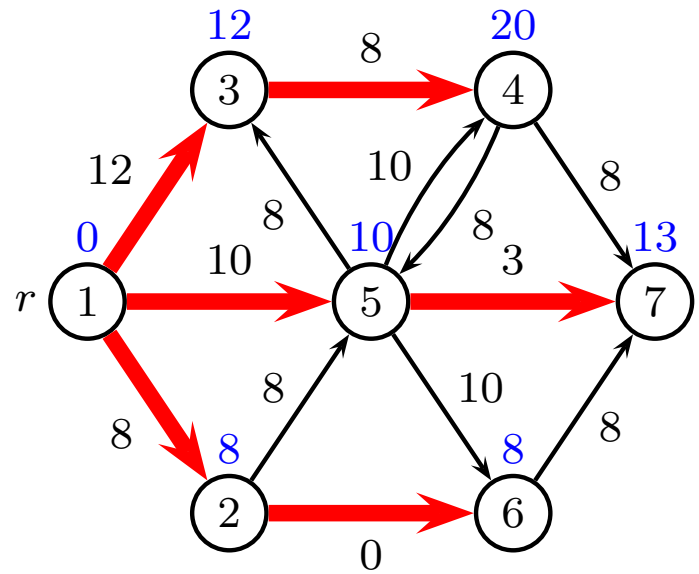
Dijkstra's Algorithm, negative edges ??

Find the most negative edge weight w_m and add $|w_m|$ to all edge weights. Now all $w_e \geq 0$. Does it work?







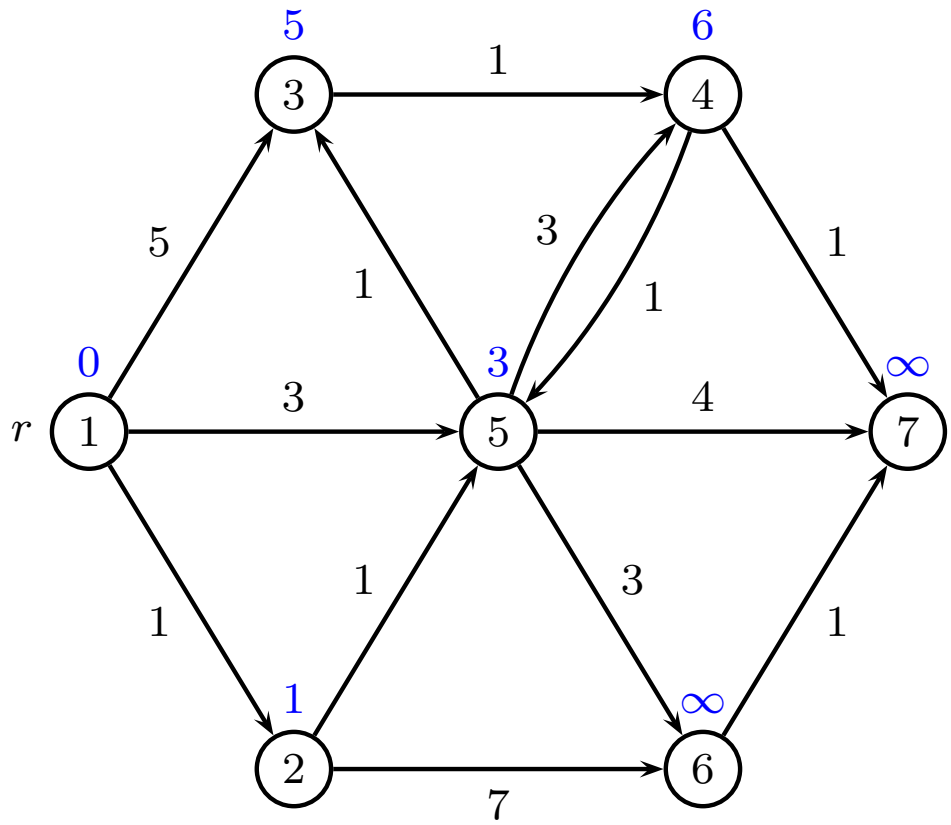


Generic shortest path

```
1 y_i ← ∞ for all i ≠ r;
2 y_r ← 0;
3 Q ← {r};
4 while Q ≠ ∅ do
5 |   select a i ∈ Q;
6 |   for {(i, j) ∈ E} do
7 |   |   if y_i + w_ij < y_j then
8 |   |   |   y_j ← y_i + w_ij;
9 |   |   |   add j to Q if not already present;
```

Q is called *candidate list* or *queue*

Example



Candidate list: $Q = [2, 5, 4]$

Label setting or label correcting

Label setting methods

- Each node is considered once
- Remove $i \in Q$ with minimum label
- Dijkstra methods
- Differ by which data structure is used to find minimum
- If arc lengths positive, only n iterations

Label correcting methods

- Each node is considered many times
- Queue Q (i.e. insert once, always remove at top)
- Many schemes for inserting $j \in Q$

Bellman-Ford

Label correcting

- Remove $i \in Q$ from top
- Insert new node j at bottom
- At most n^2 iterations

Bellman-Ford methods can be superior because of the smaller overhead per iteration!

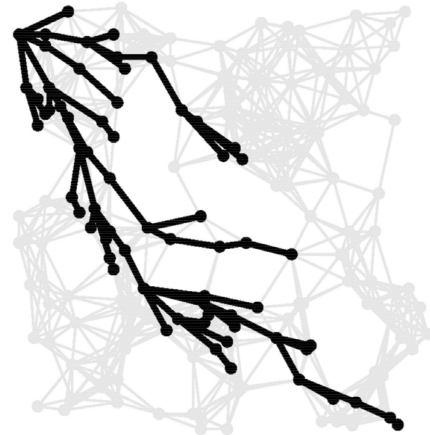
For acyclic graphs, Bellman-Ford uses n iterations, same as Dijkstra

Shortest path Euclidian graph (map)

A^* algorithm finding path from r to t

Shortest path Euclidian graph (map)

A^* **algorithm** finding path from r to t



Assume that h_i is heuristic distance from i to t.
(we can use Euclidean or Manhattan distance).

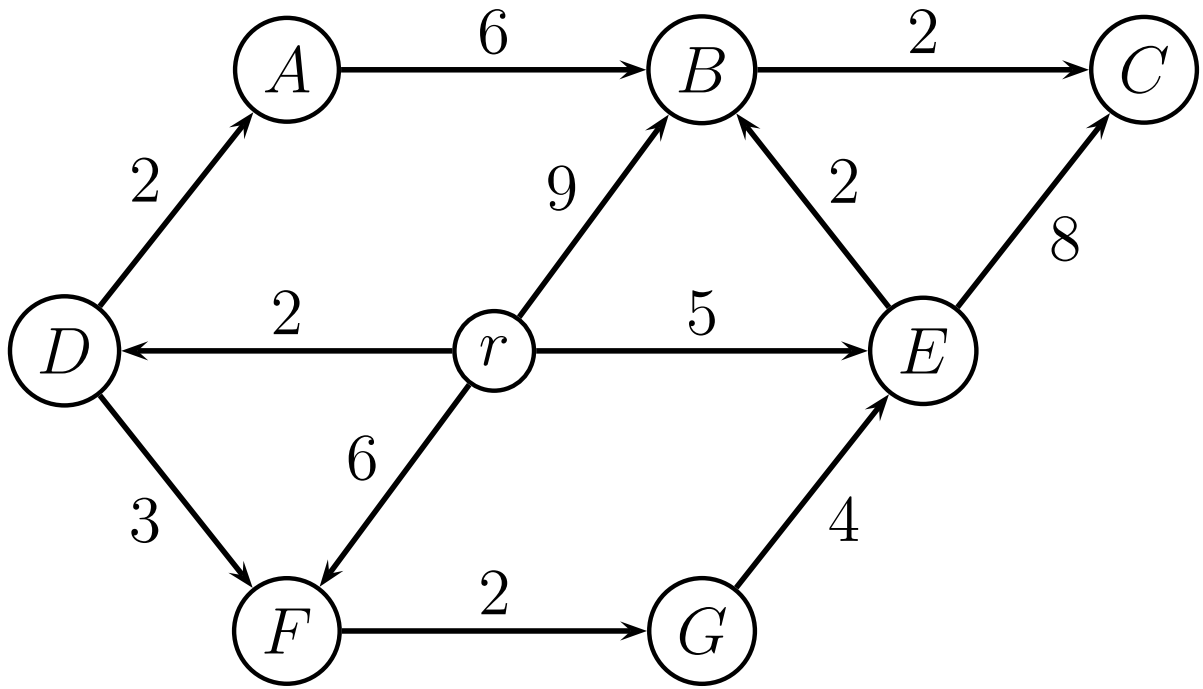
- Run Dijkstra, but selecting node with lowest $y_i + h_i$
- Stop when t is added to T
- Dijkstra's proof of correcness still holds
- In practice only $O(\sqrt{V})$ vertices examined

A^* Algorithm for Shortest Path

Let h_i be a heuristic distance from i to t
Run Dijkstra, but selecting node i with lowest $y_i + h_i$

```
1  T ← ∅;
2  y_r ← 0; y_i ← ∞ for all other i;
3  while V \ T ≠ ∅ do
4      select an i ∈ V \ T minimizing y_i + h_i;
5      for {(i, j) ∈ E : j ∉ T} do
6          if y_i + w_ij < y_j then
7              y_j ← y_i + w_ij;
8      T ← T ∪ {i};
9      if i = t then
10         stop
```

A^* Algorithm for Shortest Path, example



A^* Algorithm for Shortest Path, example

Normal Dijkstra: y_i values

node	r	A	B	C	D	E	F	G
init	**0**	∞	∞	∞	∞	∞	∞	∞
r			9		**2**	**5**	6	
D		**4**					**5**	
A								
F								**7**
E			**7**	13				
G								
B				**9**				
C								

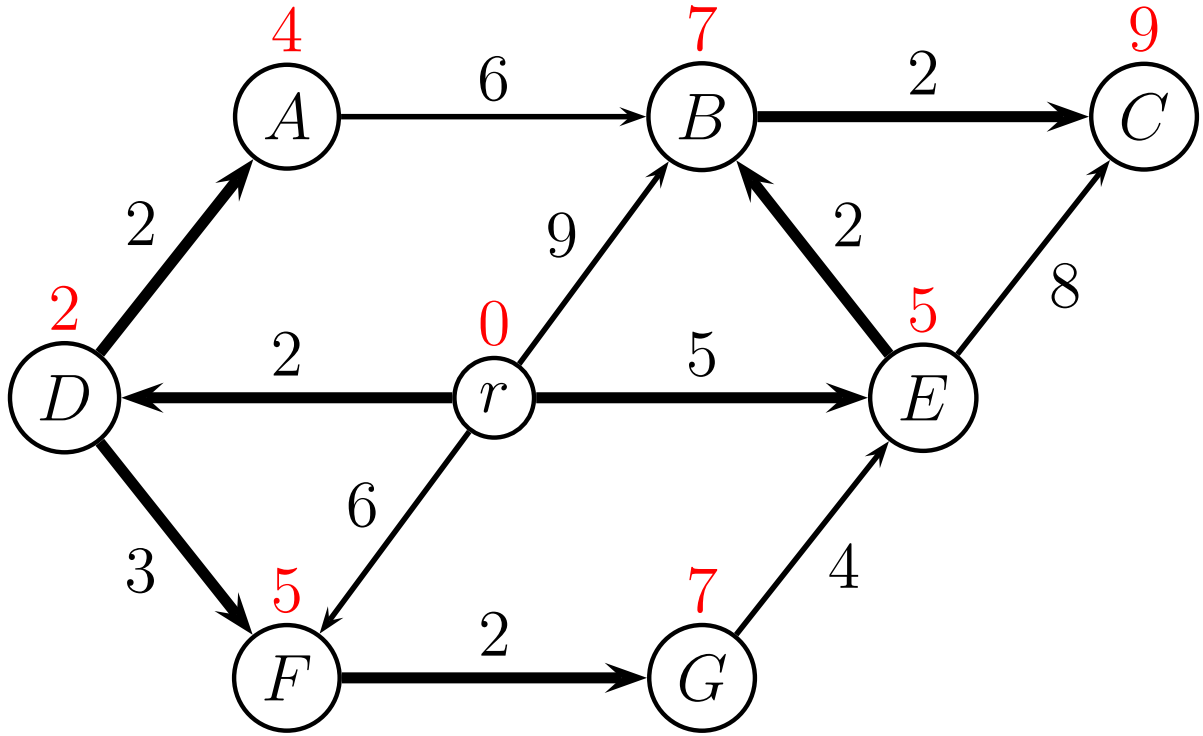
Shortest distances: marked with bold

Shortest path tree: bold numbers find when was updated

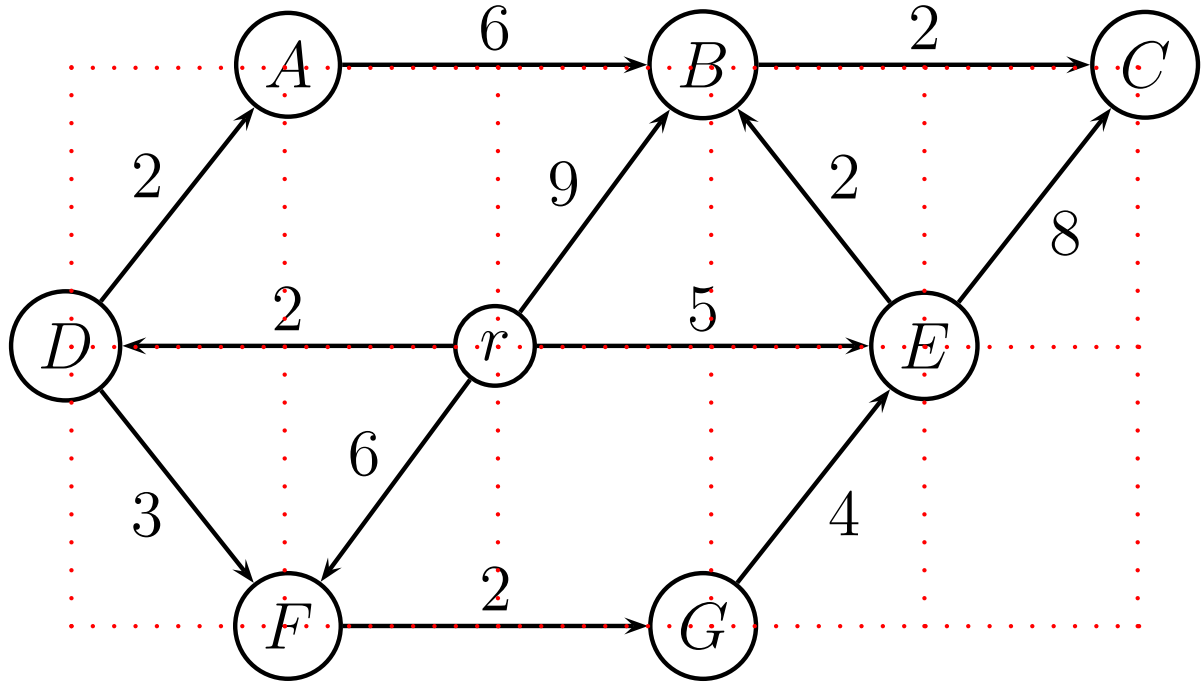
- e.g. node C (distance 9) was updated from B

A^* Algorithm for Shortest Path, example

Normal Dijkstra: shortest path tree



A^* Algorithm for Shortest Path, example



Manhattan distances to $t = C$

i	r	A	B	C	D	E	F	G
h_i	4	4	2	0	6	2	6	4

A^* Algorithm for Shortest Path, example

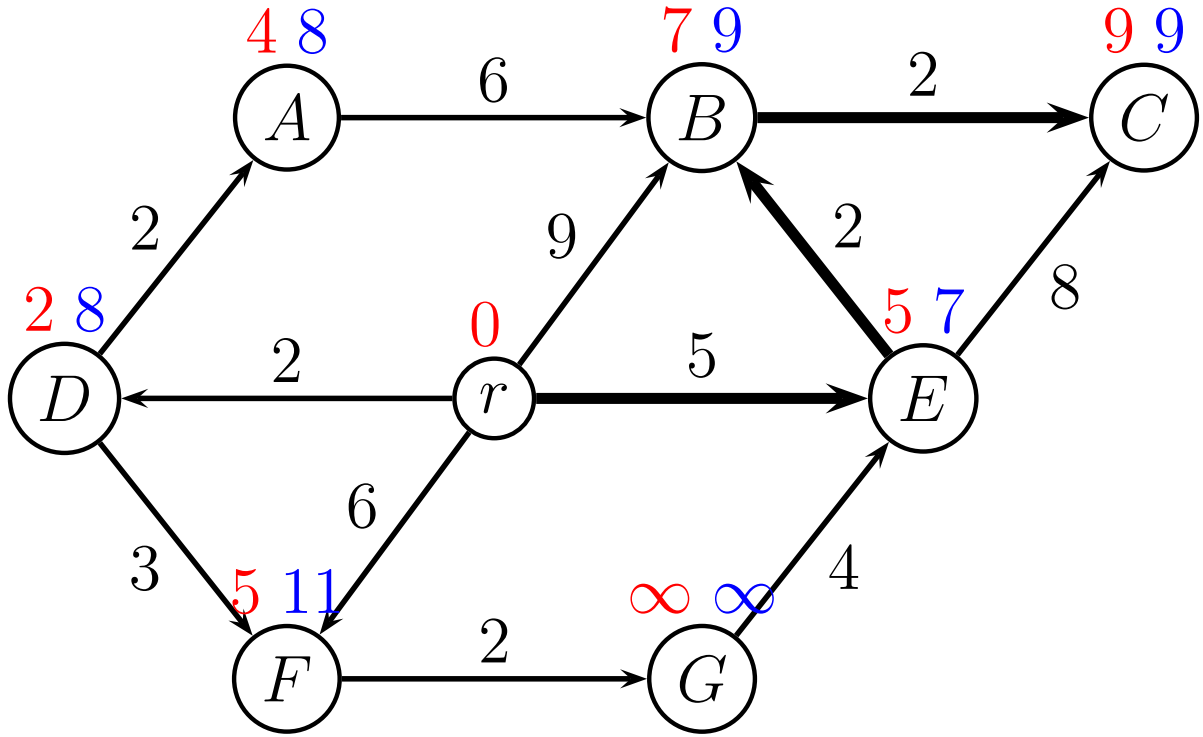
A^* algorithm: y_i and $y_i + h_i$ values

node	r	A		B		C		D		E		F		G	
h_i	4		4		2		0		6		2		6		4
init	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
r				9	11			2	8	**5**	7	6	12		
E				**7**	9	13	13								
D		4	8									5	11		
A															
B						**9**	9								
C															

Shortest path tree: Predecessor of C is B.
Predecessor of B is E. Predecessor of E is r.
The shortest path is $r \to E \to B \to C$, cost $5 + 2 + 2 = 9$.

A^* Algorithm for Shortest Path, example

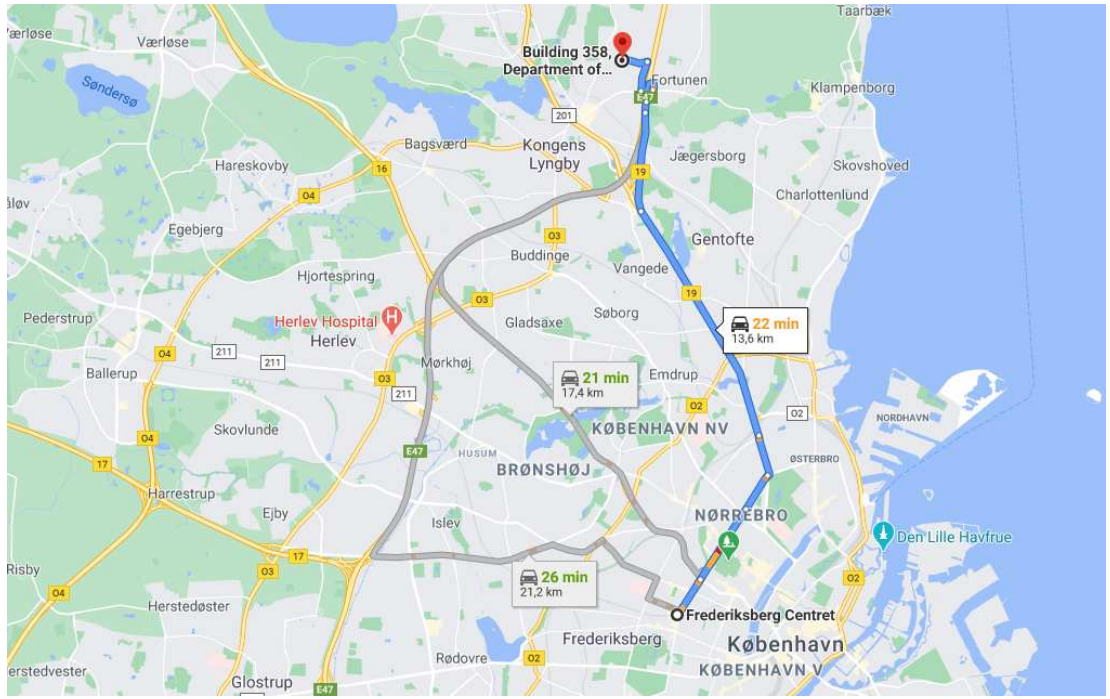
A^* algorithm: shortest path



i	r	A	B	C	D	E	F	G
h_i	4	4	2	0	6	2	6	4

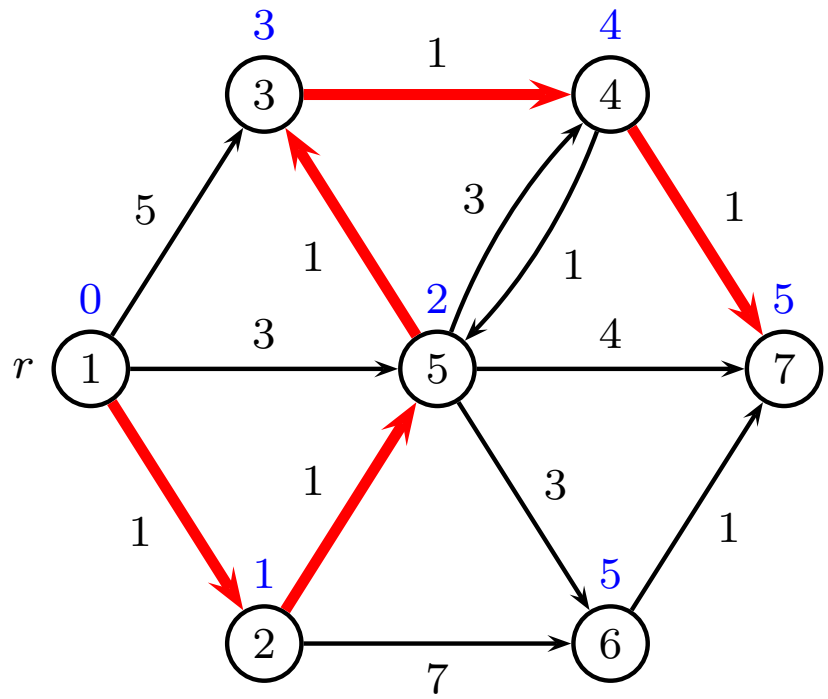
Finding the second-best path

Google maps: find alternatives



Column generation, forbidden column

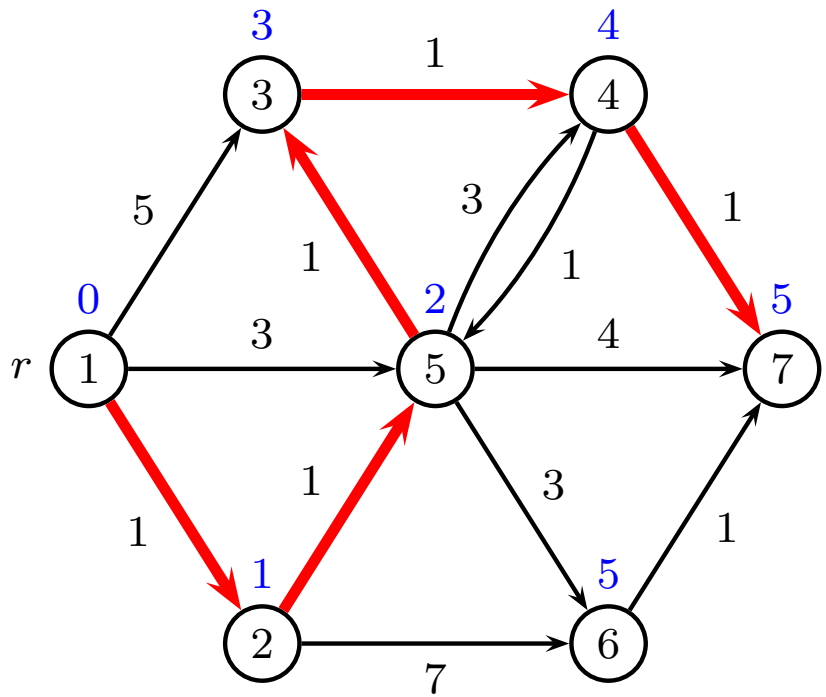
Finding the second-best path



Repeat for all edges (i, j) on shortest path
- set $w_{ij} = \infty$
- solve shortest-path problem
- set w_{ij} to old value

Select shortest path among all candidates

Finding the k best paths

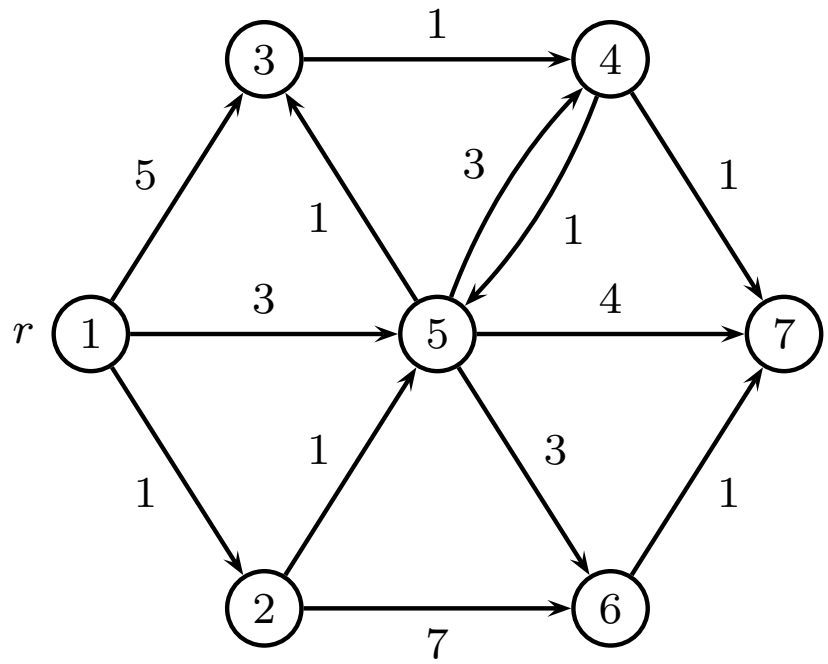


Same principle, but now multiple edges may be forbidden
Explore a branch-and-bound tree of forbidden edges

However, paths will often be very similar

Shortest-path diversification

Edge weight randomization

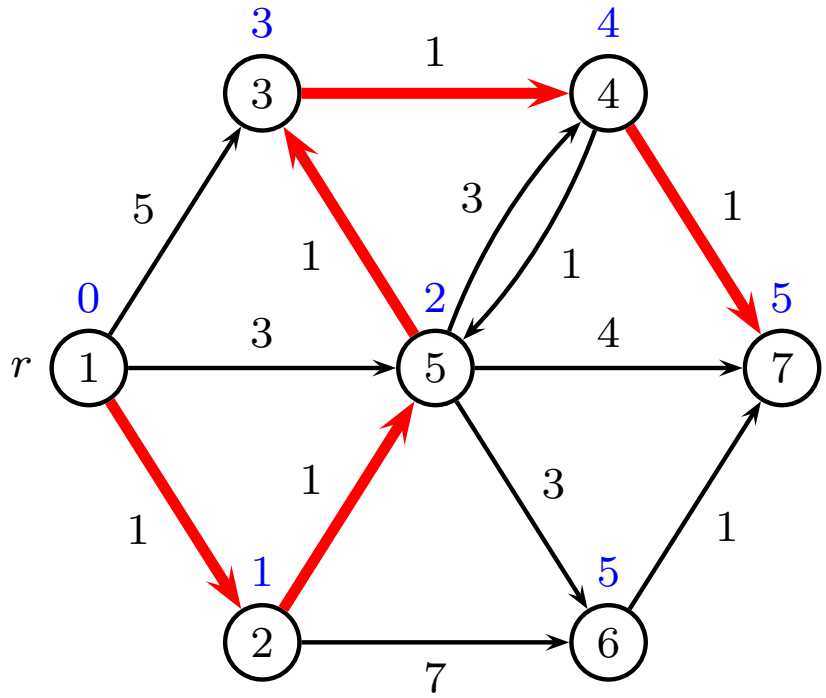


Repeat k times
- for all edges set $w_{ij} := w_{ij} +$ noise
- solve shortest-path problem

Return k candidates

Shortest-path diversification

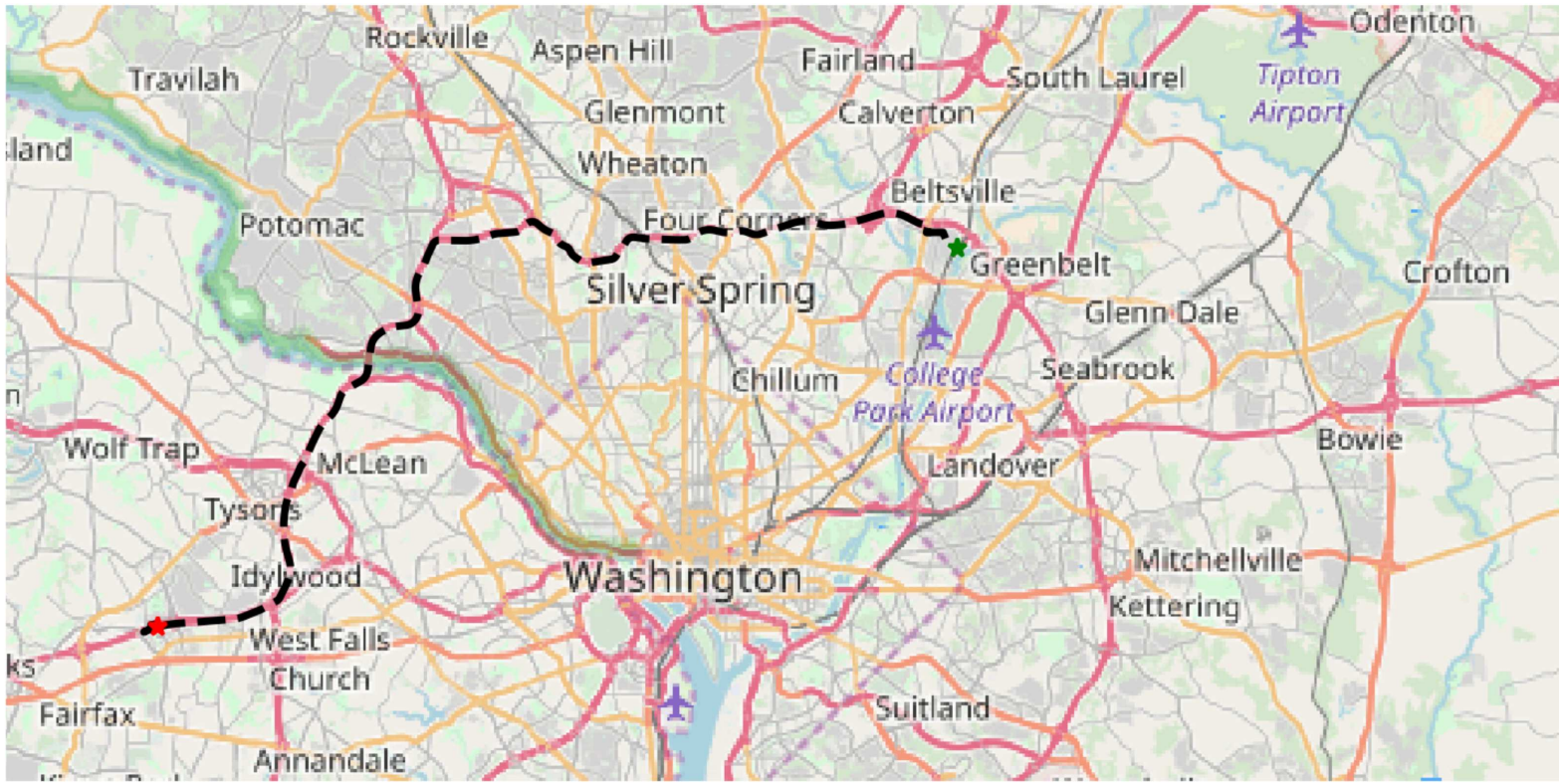
Edge weight penalization



Penalize edges on shortest path

Shortest path diversification

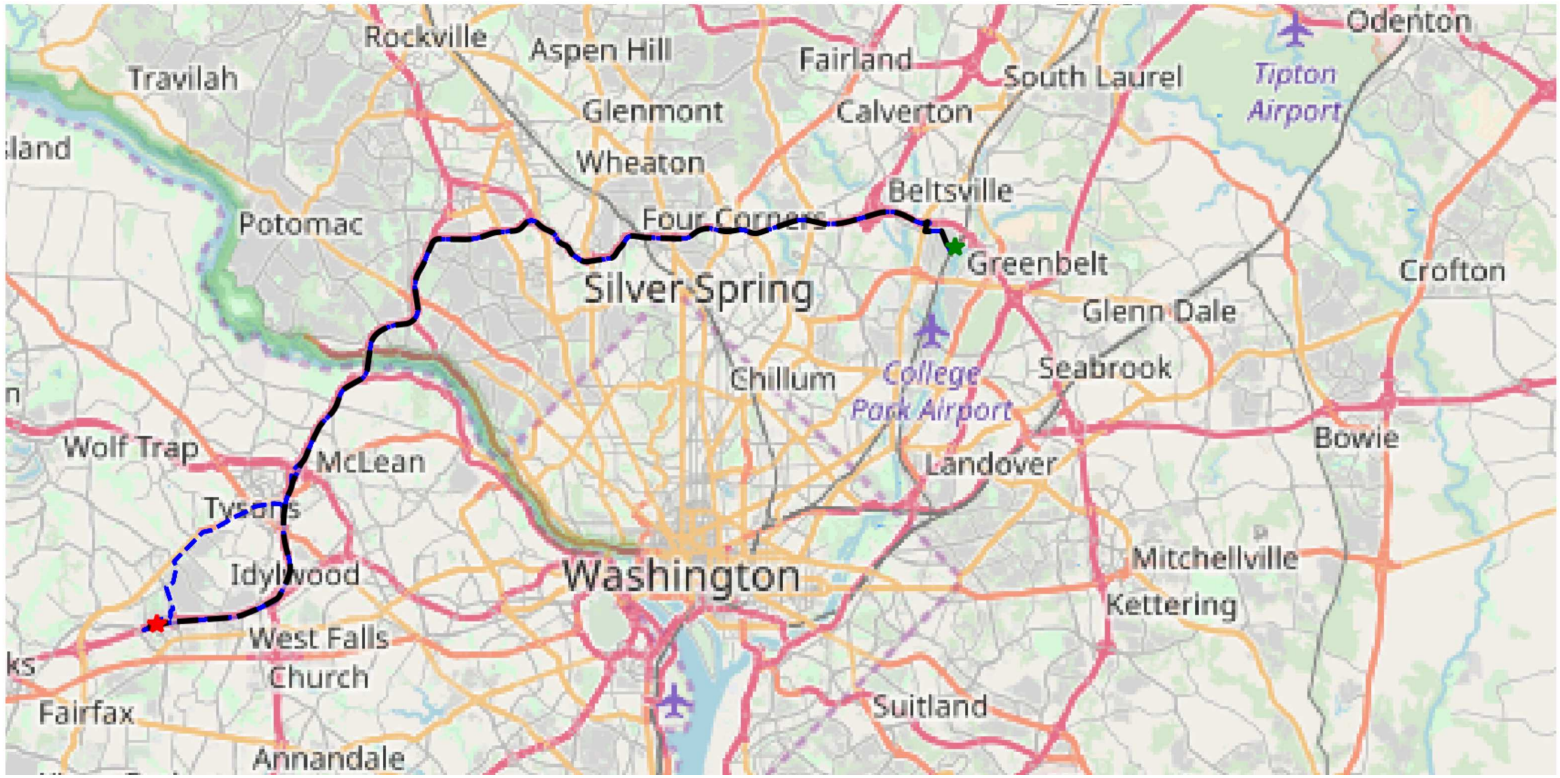
Shortest path using Dijkstra



Cheng, Gkountouna, Zufle, Pfoser, Wenk "Shortest-path diversification through network penalization: a washington DC area case study"

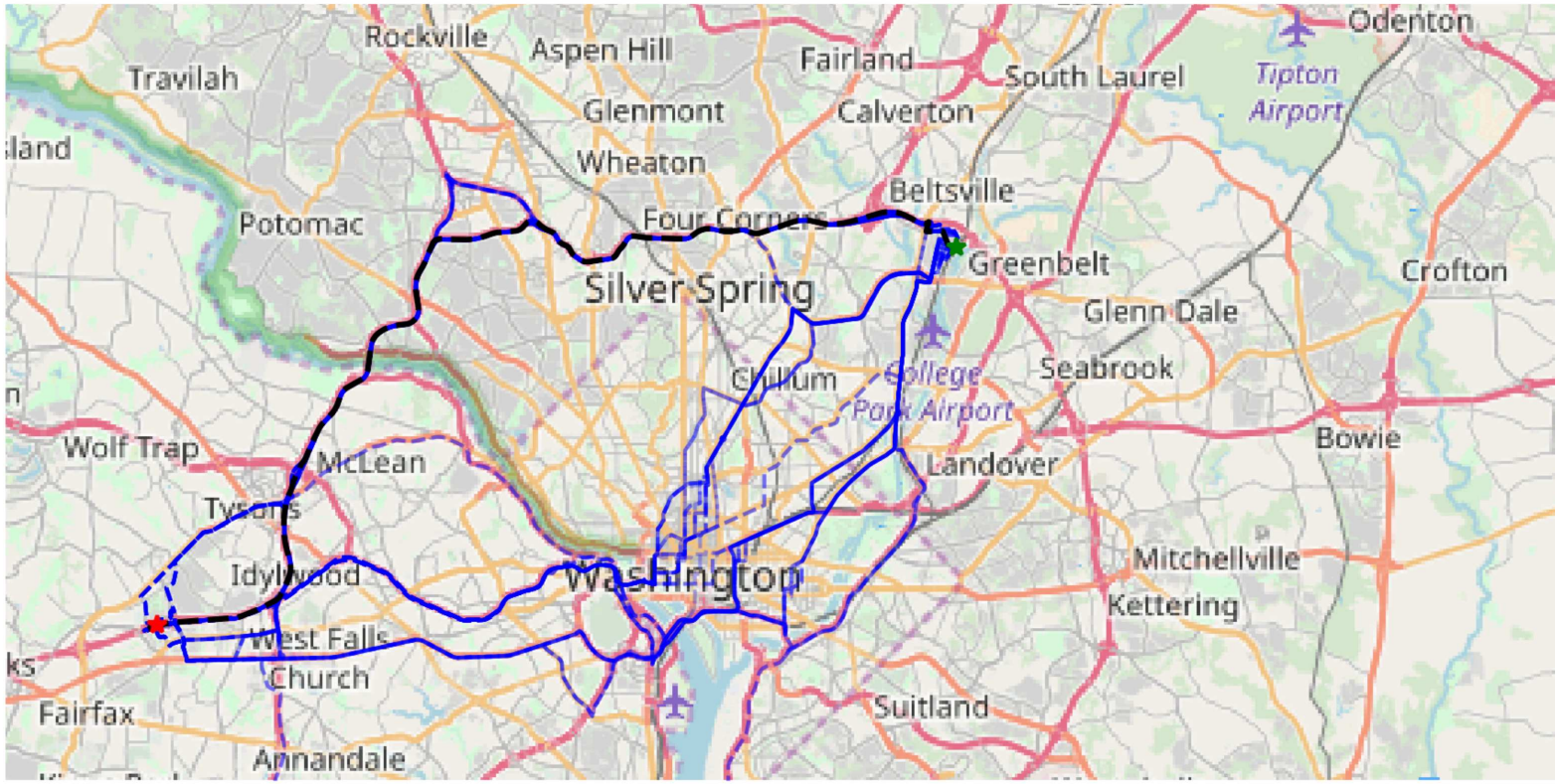
Shortest path diversification

100 path using graph randomization ($\delta = 0.1$)



Shortest path diversification

100 path using penalization ($\rho = 0.01$)



Shortest path diversification

100 path using penalization ($\rho = 0.1$)